

SYSTEM DESIGN

Comprehensive Software Engineering Interview Learning Roadmap

A structured roadmap covering system-design fundamentals, distributed systems, scalability, reliability, security, architecture, and real-world design exercises.

Roadmap Overview

#	Topic	Core areas
1	What is System Design?	Functional requirements, Non-functional requirements, Scalability, Reliability...
2	Client → Server → Database	Client-server architecture, Request/response lifecycle, Application servers, Stateless vs stateful services...
3	HTTP / HTTPS / DNS	HTTP methods and status codes, HTTPS/TLS, DNS resolution, TCP/IP basics...
4	APIs & REST	REST principles, Resource modeling, Pagination, Filtering and sorting...
5	Latency / Throughput / Bandwidth	Latency, Throughput, Bandwidth, Response-time percentiles...
6	Vertical vs Horizontal Scaling	Vertical scaling, Horizontal scaling, Elasticity, Stateless scaling...
7	Load Balancers	Reverse proxies, L4 vs L7, Round robin, Weighted routing...
8	Databases	SQL databases, NoSQL databases, Indexes, Transactions and ACID...
9	Caching	Cache hit/miss, TTL, LRU and eviction, Cache-aside...
10	CDNs	Edge servers, Origin servers, Static-content caching, Geographic distribution...
11	Message Queues	Producers and consumers, Workers, Acknowledgements, Retries...
12	Event-Driven Architecture	Events, Publish/subscribe, Event streams, Event consumers...
13	Distributed Systems	Partial failure, Network partitions, Replication, Distributed state...
14	CAP & Consistency	CAP theorem, Consistency, Availability, Partition tolerance...
15	Fault Tolerance	Redundancy, Failover, Retries, Timeouts...
16	Rate Limiting	Fixed window, Sliding window, Token bucket, Leaky bucket...
17	Authentication & Security	Authentication vs authorization, Sessions and cookies, JWT, OAuth 2.0...
18	Microservices	Monolith vs microservices, Service boundaries, API gateways, Service discovery...
19	Observability	Logs, Metrics, Distributed tracing, Latency and error rates...
20	Real System Designs	URL Shortener, Chat Application, Uber / Ride Sharing, YouTube / Video Platform...

Detailed Curriculum

1. What is System Design?

- Functional requirements
- Non-functional requirements
- Scalability
- Reliability
- Maintainability
- Performance
- Cost and trade-offs

2. Client → Server → Database

- Client-server architecture
- Request/response lifecycle
- Application servers
- Stateless vs stateful services
- Three-tier architecture

3. HTTP / HTTPS / DNS

- HTTP methods and status codes
- HTTPS/TLS
- DNS resolution
- TCP/IP basics
- Cookies and sessions

4. APIs & REST

- REST principles
- Resource modeling
- Pagination
- Filtering and sorting
- API versioning
- Idempotency
- GraphQL, gRPC, WebSockets

5. Latency / Throughput / Bandwidth

- Latency
- Throughput
- Bandwidth
- Response-time percentiles
- Performance bottlenecks

- Capacity estimation

6. Vertical vs Horizontal Scaling

- Vertical scaling
- Horizontal scaling
- Elasticity
- Stateless scaling
- Read-heavy vs write-heavy workloads

7. Load Balancers

- Reverse proxies
- L4 vs L7
- Round robin
- Weighted routing
- Least connections
- Health checks
- Sticky sessions
- Failover

8. Databases

- SQL databases
- NoSQL databases
- Indexes
- Transactions and ACID
- Isolation levels
- Replication
- Read replicas
- Sharding
- Partitioning
- Normalization and denormalization

9. Caching

- Cache hit/miss
- TTL
- LRU and eviction
- Cache-aside
- Write-through
- Write-back
- Distributed caching
- Cache invalidation

- Cache stampede
- Hot keys

10. CDNs

- Edge servers
- Origin servers
- Static-content caching
- Geographic distribution
- Cache invalidation
- CDN vs application cache

11. Message Queues

- Producers and consumers
- Workers
- Acknowledgements
- Retries
- Dead-letter queues
- Ordering
- Idempotency
- Backpressure
- Kafka, RabbitMQ, SQS

12. Event-Driven Architecture

- Events
- Publish/subscribe
- Event streams
- Event consumers
- Loose coupling
- Event ordering
- Duplicate events
- Eventual consistency

13. Distributed Systems

- Partial failure
- Network partitions
- Replication
- Distributed state
- Consistency
- Availability
- Fault domains

- Service communication

14. CAP & Consistency

- CAP theorem
- Consistency
- Availability
- Partition tolerance
- CP vs AP
- Strong consistency
- Eventual consistency
- Read-after-write consistency
- Causal consistency

15. Fault Tolerance

- Redundancy
- Failover
- Retries
- Timeouts
- Circuit breakers
- Graceful degradation
- Bulkheads
- Disaster recovery
- SLA, SLO, SLI
- MTTR and MTBF

16. Rate Limiting

- Fixed window
- Sliding window
- Token bucket
- Leaky bucket
- Distributed rate limiting
- Redis-based rate limiting
- Fairness and abuse prevention

17. Authentication & Security

- Authentication vs authorization
- Sessions and cookies
- JWT
- OAuth 2.0
- OpenID Connect

- Access/refresh tokens
- Password hashing
- MFA
- TLS
- Encryption
- RBAC
- Least privilege
- OWASP threats

18. Microservices

- Monolith vs microservices
- Service boundaries
- API gateways
- Service discovery
- Inter-service communication
- Data ownership
- Distributed transactions
- Saga pattern
- Operational complexity

19. Observability

- Logs
- Metrics
- Distributed tracing
- Latency and error rates
- Correlation IDs
- Dashboards
- Alerts
- Debugging production systems

20. Real System Designs

- URL Shortener
- Chat Application
- Uber / Ride Sharing
- YouTube / Video Platform
- Instagram / Social Feed
- E-commerce Platform

The System Design Interview Framework

1. Requirements

Clarify what the system must do and identify functional and non-functional requirements.

2. Scale Estimation

Estimate users, requests/second, storage, bandwidth, concurrency, and growth.

3. API Design

Define the major endpoints, inputs, outputs, pagination, errors, and idempotency.

4. Data Model

Choose entities, relationships, indexes, storage technology, and access patterns.

5. High-Level Architecture

Draw the major components and explain how requests and data flow through them.

6. Scaling

Identify bottlenecks and decide where to use caching, replication, partitioning, queues, or CDNs.

7. Reliability

Discuss redundancy, retries, timeouts, failover, degradation, and disaster recovery.

8. Security

Cover authentication, authorization, encryption, secrets, rate limits, and common attack surfaces.

9. Bottlenecks

Identify what will fail first as traffic grows and explain how you would address it.

10. Trade-offs

Explicitly compare alternatives and justify the design choices.

Core Mental Model

Requirements → Scale → Bottleneck → Component → Trade-off → Failure Mode → Solution

Recommended Practice Progression

Beginner

URL Shortener, Pastebin, File Storage, Rate Limiter, Voting System, Library Management

Intermediate

Chat Application, Notification System, Ride Sharing, Food Delivery, E-commerce, Payment System, Job Platform, Social Feed

Advanced

YouTube, Netflix, WhatsApp, Instagram, Twitter/X, Google Drive, Uber, Google Maps, Distributed Web Crawler, Search Engine, Analytics Platform

Study Method

- Learn the concept and the problem it solves.
- Understand how the component works internally.
- Draw the architecture from memory.
- Learn the trade-offs and failure modes.
- Implement small pieces where useful using Python or Node.js.
- Practice explaining decisions aloud as if in an interview.
- After each design, identify bottlenecks and redesign the system for 10× and 100× growth.

Goal: By the end of this roadmap, you should be able to approach an unfamiliar system-design problem and reason from requirements to architecture rather than simply recalling a memorized diagram.